

Infrastructure & Terraform Guide

Part 1: AWS Core Concepts

1. S3 (Simple Storage Service)

Theory: S3 is an object storage service. It is bucket-based, where each object consists of data, metadata, and a key. It offers high durability (99.999999999%) and availability.

Basic Policy: S3 policies control access. User-based (IAM) policies attach to users; Resource-based (Bucket) policies attach to the bucket itself. A standard public read policy restricts access to your bucket unless explicitly allowed.

2. Site-to-Site VPN

Theory: Connects your on-premises network to an AWS VPC. It involves a **Virtual Private Gateway** (on the AWS side) and a **Customer Gateway** (on your side). It establishes encrypted tunnels over the public internet, using IPsec for secure communication and BGP for route propagation.

Part 2: Terraform Fundamentals

3. Terraform CLI Lifecycle

- **init:** Initializes the working directory (downloads providers/modules).
- **validate:** Checks configuration for syntax errors.
- **fmt:** Formats code into canonical format.
- **plan:** Creates an execution plan (preview changes).
- **apply:** Executes the planned changes.
- **destroy:** Deletes all resources managed by the state file.

4. Terraform Blocks

- **provider:** Defines the cloud environment (e.g., aws, azurerm).
- **variable:** Defines input variables to make code reusable.

- **resource:** Defines the actual infrastructure object (e.g., `aws_instance`).
- **output:** Exports information needed by other modules or the user.

5. Terraform Registry & EC2

The Registry is a central hub for shared modules. To use an EC2 module:

```
module "ec2_instance" { source = "terraform-aws-modules/ec2-instance/aws"
```

Part 3: Advanced Terraform

6. Meta-Arguments

- **count:** Creates multiple instances of a resource based on an integer.
- **for_each:** Creates multiple instances based on a map or set of strings.
- **depends_on:** Handles implicit dependencies not automatically detected.
- **lifecycle:** Controls `create_before_destroy`, `prevent_destroy`, and `ignore_changes`.

7. Network Components via Terraform

Typical automation includes: `aws_vpc`, `aws_subnet` (public/private), `aws_internet_gateway`, `aws_route_table`, and `aws_security_group`. These form the skeletal backbone of your cloud infra.

8. Modules

Modules encapsulate resources. **Parent** calls the modules; **Child** is the module being called. **Sibling** refers to modules referencing each other within the same directory structure.

9. 3-Tier Infra

Without Meta-handler: Declaring every instance individually (Web1, Web2, DB1, DB2).

With Meta-handler: Using `for_each` or `count` to scale instances (e.g., 3 web servers) dynamically with a single block, reducing code duplication.

Part 4: EKS Overview

10. EKS (Elastic Kubernetes Service)

- **AWS VPC CNI:** The plugin that allows Kubernetes pods to get IP addresses from the VPC network.
 - **AWS LB Controller:** Provisions AWS Elastic Load Balancers (ALB/NLB) for Kubernetes Ingress/Services.
 - **IRSA (IAM Roles for Service Accounts):** Maps IAM roles to K8s service accounts, allowing specific pods (not the whole node) to access AWS services (like S3/RDS).
 - **OIDC (OpenID Connect):** An identity provider configuration that allows Kubernetes to talk to AWS IAM.
 - **EBS Storage Driver:** The CSI (Container Storage Interface) plugin that allows pods to provision and attach AWS EBS volumes.
-